

FoodieGo

Technical Specification & Deployment Manual

Cloud Computing Capstone Project

Level 4 — IS Department

Student ID	Student Name
2022170043	أحمد مصطفى عبدالراضي حسين
2022170595	كمال الدين سيد كمال عودة
2022170283	عمر محمد سيد احمد
2022170265	عمر احمد عباس عبدالعليم
2022170457	ميناء توفيق حمادة
2022170590	عمر محمد وفيق ذكي محمد العطار

Contents

1	Executive Summary	2
1.1	Project Vision	2
2	System Architecture & Philosophy	3
2.1	Architectural Pattern: Microservices	3
2.2	Communication Strategy	3
2.3	Database-per-Service Simulation	3
3	Core Components Deep-Dive	4
3.1	Frontend Application	4
3.2	Backend Services	4
3.3	Async Processing & Integrity	4
4	Infrastructure & Orchestration	5
4.1	Dockerization Strategy	5
4.2	Multi-Environment Strategy	5
4.3	Kubernetes (K8s) Deployment	5
5	State-of-the-Art Observability	6
5.1	Monitoring Stack	6
5.2	Centralized Logging	6
5.3	Visual Evidence & Dashboards	6
5.3.1	Cluster Health Monitoring	6
5.3.2	Log Aggregation and Analysis	7
6	Setup & Deployment Guide	8
6.1	Kubernetes Production Build	8
7	Quality Assurance	9
7.1	Testing Strategy	9
8	Conclusion	10

Chapter 1

Executive Summary

1.1 Project Vision

FoodieGo is a high-performance food delivery platform engineered to demonstrate advanced cloud-native paradigms. The system provides a seamless digital marketplace for customers, restaurant owners, and delivery drivers. Architecturally, it prioritizes horizontal scalability, fault isolation, and full-stack observability.

The project highlights three core pillars of modern cloud engineering:

- **Decoupled Microservices:** Domain-driven backend isolation.
- **Event-Driven Logic:** Asynchronous processing via RabbitMQ message brokering.
- **DevOps Automation:** Container orchestration with Kubernetes and centralized observability with Prometheus, Grafana, and Loki.

Chapter 2

System Architecture & Philosophy

2.1 Architectural Pattern: Microservices

Transitioning from a monolithic approach, FoodieGo utilizes four independent backend services. This pattern allows for independent deployment cycles and technology agility.

Service	Port	Core Responsibility
User Service	3001	IAM, Registration, JWT Auth, and Driver Lookups.
Restaurant Service	3002	Catalog management, Menu CRUD, and real-time availability.
Order Service	3003	Orchestrator for order state management and price integrity.
Payment Service	3004	Async background worker for financial ledger recording.

2.2 Communication Strategy

The platform employs a **Polyglot Communication** strategy:

- **Synchronous (REST/Axios):** Used for immediate validation. For example, the Order Service calls the User Service to verify identity and the Restaurant Service to fetch current menu prices.
- **Asynchronous (RabbitMQ):** Used for non-blocking workflows. Order-to-Payment triggers are offloaded to a queue, ensuring the user interface remains responsive while processing happens in the background.

2.3 Database-per-Service Simulation

To adhere to microservices best practices without infrastructure overhead, we utilize a single PostgreSQL 16 instance partitioned into **4 logical schemas**: `user_svc`, `restaurant_svc`, `order_svc`, and `payment_svc`.

Chapter 3

Core Components Deep-Dive

3.1 Frontend Application

The UI is a single-page application (SPA) built with **React 18** and **Vite 5**. In production, the application is bundled into optimized static files and served via **Nginx**, which acts as a reverse proxy for all `/api/*` traffic.

3.2 Backend Services

All services are built on **Node.js 20**. Each service is instrumented with `prom-client` to expose a native `/metrics` endpoint, providing real-time data on Heap usage and Event Loop lag.

3.3 Async Processing & Integrity

Payment processing is handled via RabbitMQ. To prevent data loss, we implement **Durable Queues** and **Dead Letter Queues (DLQ)**, where failed messages are routed for manual administrative review.

Chapter 4

Infrastructure & Orchestration

4.1 Dockerization Strategy

We utilize multi-stage builds to optimize security and image size:

- **Backend (2-stage):** Stage 1 installs production dependencies; Stage 2 bundles only the runtime into an Alpine-based image.
- **Frontend (3-stage):** Development → Vite Build → Nginx Production.

4.2 Multi-Environment Strategy

The project provides three tailored Docker Compose configurations:

- **Dev:** Mounts local `./src` for hot-reloading with `nodemon`.
- **Test:** Uses separate database containers (`food_delivery_test`) for isolated assertions.
- **Prod:** Hardened environment with strict environment variable enforcement.

4.3 Kubernetes (K8s) Deployment

The system is orchestrated using **26 Kubernetes manifests**, utilizing **Deployments** for stateless code and **StatefulSets** for data-sensitive services like Postgres, RabbitMQ, and Loki.

Chapter 5

State-of-the-Art Observability

5.1 Monitoring Stack

The system features zero-config monitoring. Prometheus utilizes **K8s Service Discovery** to scrape any pod annotated with `prometheus.io/scrape`.

5.2 Centralized Logging

We implemented the **Loki + Promtail** stack. Promtail runs as a **DaemonSet** on every node, tailing pod logs from `/var/log/pods` and shipping them to Loki for persistent storage.

5.3 Visual Evidence & Dashboards

5.3.1 Cluster Health Monitoring

The Grafana dashboard illustrates real-time resource utilization across the cluster, including pod counts, CPU/Memory per service, and Node.js heap memory statistics.

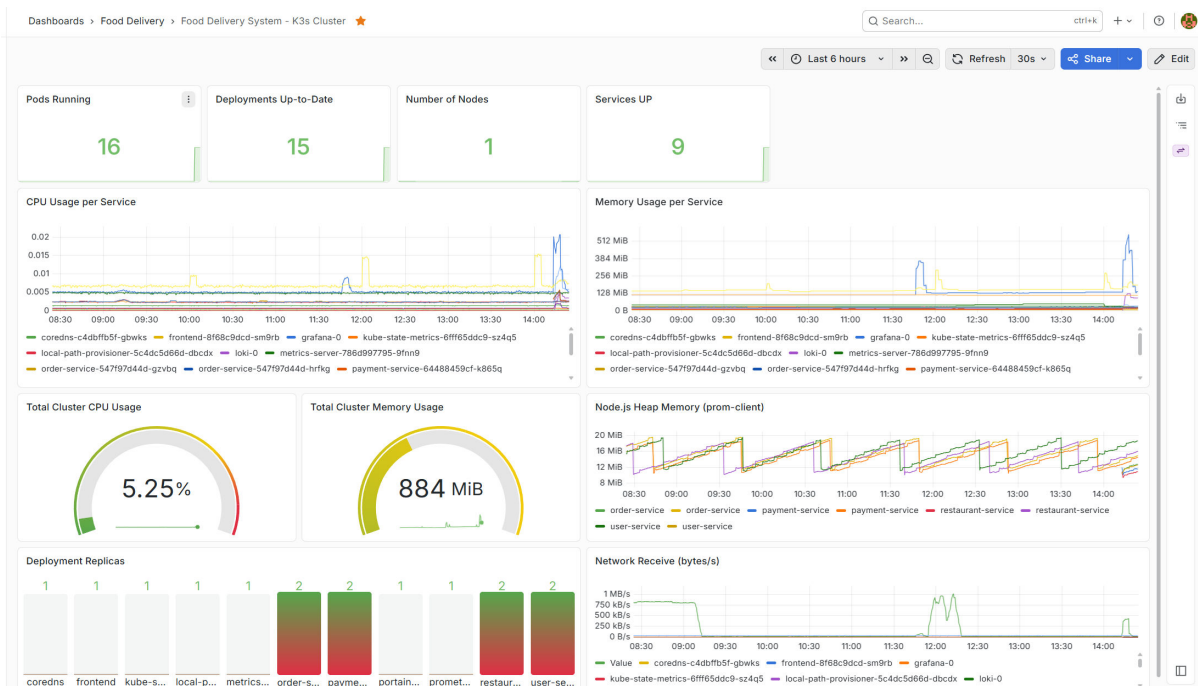


Figure 5.1: Pre-provisioned Grafana Dashboard showing cluster-wide metrics.

5.3.2 Log Aggregation and Analysis

Centralized log exploration using Loki allows developers to query stdout/stderr across all microservices using LogQL, significantly reducing troubleshooting time.

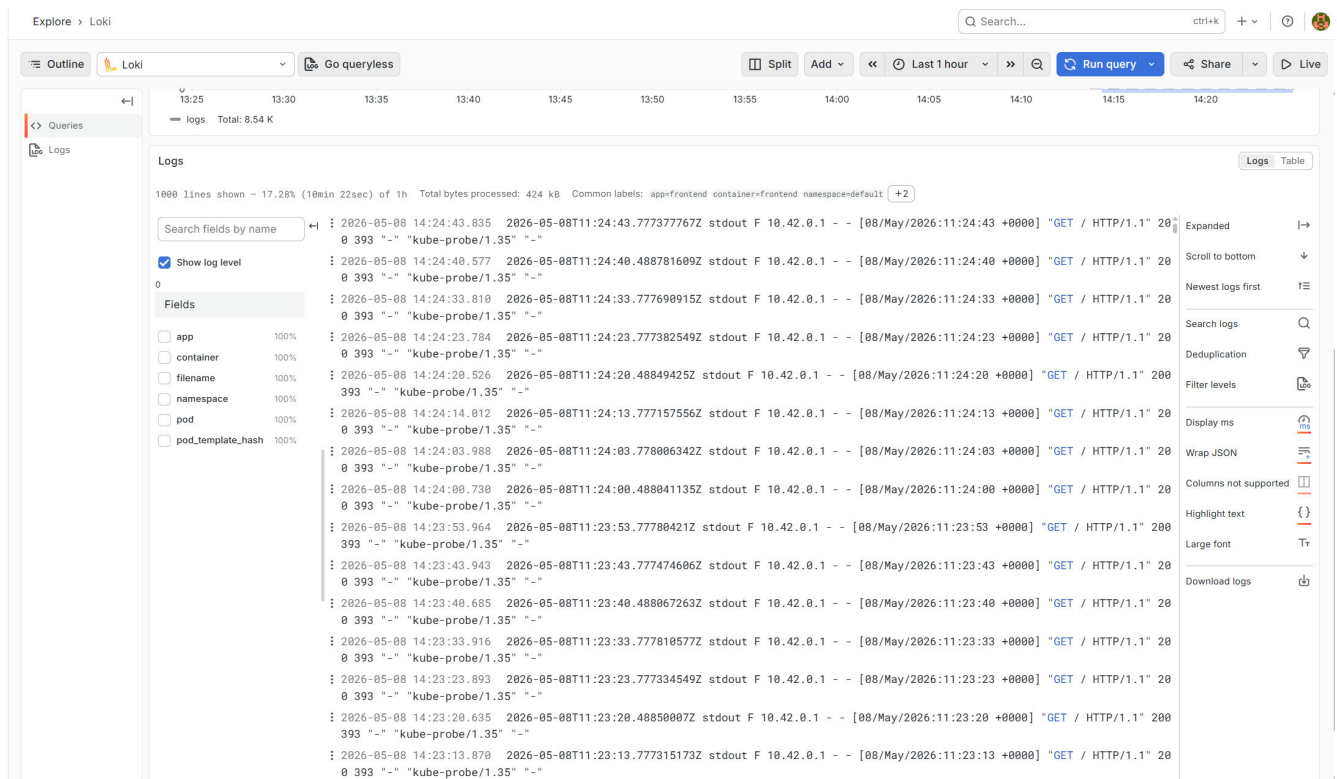


Figure 5.2: Centralized log collection and querying using Loki.

Chapter 6

Setup & Deployment Guide

6.1 Kubernetes Production Build

```
1 # 1. Point to Minikube registry
2 eval $(minikube docker-env)
3
4 # 2. Build core images
5 docker build -t food-delivery/user-service:latest ./services/user-service
6 docker build -t food-delivery/order-service:latest ./services/order-service
7 # ... other services ...
8
9 # 3. Apply all manifests
10 kubectl apply -f k8s/
```

Chapter 7

Quality Assurance

7.1 Testing Strategy

- **Unit Tests:** Jest-based suites for individual service logic.
- **E2E Suite:** A master script (`run-e2e.js`) performs **44 assertions** against the live stack, covering the full user journey.

Chapter 8

Conclusion

FoodieGo successfully applies modern cloud paradigms to solve real-world architectural challenges. By prioritizing decoupling, orchestration, and observability, the platform provides a robust blueprint for high-availability systems.